

Pupil data processing and statistical analysis

Adrian RUIZ CHIAPELLO
CNRS, CerCo, UMR5549, Toulouse, France
Toulouse University, Toulouse, France

Pupil data processing

We introduce the pseudo-code for the pupil data processing algorithm used in this study. The algorithm was specifically developed to process raw pupil data captured by an Eyebrain T2 Tracker (SuriCog, Paris, France). The steps of the algorithm are outlined in [Algorithm 1](#), and the explanations for key steps are provided below.

Algorithm 1 Pupil data processing

- 1: **Input:** Pupil diameter time series $X = \{x_1, x_2, \dots, x_n\}$ from both eyes (x-axis)
 - 2: **Output:** Processed TEPR and pupil metrics
 - 3: Initialize window size $w = 2k + 1 = 11$, threshold $N_\sigma = 2$, scaling factor $\kappa = 1.4826$
 - 4: **for** each data point x_i in X **do**
 - 5: Extract window $W_i = \{x_{i-k}, \dots, x_{i+k}\}$ around x_i
 - 6: Compute the median $M(W_i)$ of the window
 - 7: Compute the MAD: $MAD(W_i) = \text{median}(|x_j - M(W_i)|)$ for all $x_j \in W_i$
 - 8: Compute the normalized deviation: $d_i = \frac{|x_i - M(W_i)|}{\kappa \cdot MAD(W_i)}$
 - 9: **if** $d_i > N_{sigma}$ **then**
 - 10: Mark x_i as an outlier and replace it with local median $M(W_i)$
 - 11: **else**
 - 12: Mark x_i as normal
 - 13: Assign newly filtered signal to $Y = \{y_1, y_2, \dots, y_n\}$
 - 14: Initialize 0th and 1st-order time derivative thresholds $T_0 = 100$ and $T_1 = 40$
 - 15: Compute $Y' = \{y_1 - y_0, y_2 - y_1, \dots, y_n - y_{n-1}\}$ the 1st-order time derivative of Y
 - 16: **for** $i = 1$ to n **do**
 - 17: **if** $|y_i| > T_0$ or $|y'_i| > T_1$ **then**
 - 18: Mark y_i as an blinking artifact and replace it with NaN
 - 19: **else**
 - 20: Mark y_i as normal
 - 21: Apply linear interpolation to replace NaN values
 - 22: Low-pass filter newly interpolated signal at 4Hz to remove non-physiological noise
 - 23: Detrend newly low-pass filtered signal to correct for possible drift
 - 24: Split the signal into seventy-two 5-second long segments starting at each stimulus onset
 - 25: Baseline-correct the segment by subtracting the mean signal over the 200ms before stimulus onset
 - 26: Group the segments according to their conditions (scene congruence, subsequent memory, etc.) and calculate the median at each time point
 - 27: Compute pupil metrics
-

Explanation :

1. Hampel outlier detection (lines 3-14). This step employs MATLAB's *hampel.m* function to identify and correct artifacts caused by brief interruptions in pupil measurements, which often produce extreme values. The Hampel filter, applied with a window spanning 5 adjacent points on either side of the target, effectively addresses these issues.

2. Blink artifact detection (lines 15-24). MATLAB's *diff.m* function is used to detect artifacts caused by blinks, which result in rapid and significant signal fluctuations. Both the magnitude of the signal and its rate of change (time derivative) are assessed using predefined thresholds.

3. Linear interpolation (line 25). Missing data points (marked as NaN) are replaced using MATLAB's *interp1.m* function with linear interpolation. This method avoids the overestimation issues associated with spline interpolation, particularly in cases where blink artifacts occur near peak dilation values. Indeed, some individuals, particularly older people, tend to blink more than their younger counterparts, especially right after responding by pressing the key, i.e. around the time of peaking dilation. To avoid artificially inflating the dilation peak among participants, we opted for linear interpolation.

4. Low-pass filtering (line 26). A low-pass filter, implemented via MATLAB's *lowpass.m* function, removes signal fluctuations exceeding the physiological limits of pupil dilation or constriction.

5. Detrending (line 27). MATLAB's *detrend.m* function eliminates linear drift in the signal, which may become noticeable over longer recording periods (e.g. 10 minutes).

6. Averaging TEPRs by computing the median trial (lines 28-30). MATLAB's *median.m* function calculates the median trial to reduce impact of outlying trial-wise pupillary responses and derive a representative TEPR. Trials with excessive interpolation ($> 25\%$) or aberrant baselines are excluded. For trials involving subsequent memory, only participants with a recognition performance above chance level (59%) contribute to the median response.

7. Computing all pupillary metrics (line 31). Let $X(t)$ be the median TEPR:

$$\textit{Pre-stimulus baseline diameter} = \frac{1}{0.2} \int_{-0.2}^0 X(t) dt,$$

$$\textit{dilation amplitude } X_{max} = \max(X(t)) \forall t \in [0, 5]s,$$

$$\textit{latency to dilation peak } t_{max} = t|_{X(t)=X_{max}},$$

$$\textit{dilation velocity} = \frac{X_{max}}{t_{max}},$$

$$\textit{mean dilation} = \frac{1}{5} \int_0^5 X(t)|_{X(t)>0} dt.$$

Cluster-based permutation testing (CBPT)

CBPT addresses the question *what is the probability that we find a cluster the size we observed, under the assumption that the null hypothesis H_0 is true?* This pseudo-code illustrates the CBPT method, which was first used in this study before switching to TFCE (see below). This algorithm was designed to assess statistical significance in task-evoked pupillary responses (TEPR) between two conditions (paired comparison) as well as between two age groups (unpaired comparison). The steps of the algorithm are outlined in [Algorithm 2](#), and the explanations for key steps are provided below.

Algorithm 2 Cluster-based Permutation Testing for 1D Pupil Signal

- 1: **Input:** Pupil signal $S_1(t)$ for condition 1, $S_2(t)$ for condition 2, number of permutations $N_{perm} = 50\,000$, significance threshold $\alpha = 0.025$, test direction (one or two-tailed), data dependence (paired or unpaired)
 - 2: **Output:** Significant clusters of time points
 - 3: Compute observed t-statistics $T_{obs}(t)$ for each time point t between S_1 and S_2
 - 4: Compute degree of freedom df
 - 5: **if** data dependence = unpaired **then**
 - 6: $df = \text{size}(S_1(t) + S_2(t)) - 2$
 - 7: **else if** data dependence = paired **then**
 - 8: $df = \text{size}(S_1(t)) - 1$
 - 9: Identify clusters C_{obs} in $T_{obs}(t)$ where t-values exceed a critical threshold $T_{crit} = \text{tinv}(1-\alpha, df)$ (or $\text{tinv}(1 - \alpha/2, df)$ if two-tailed) according to df and α :
 - 10: **if** test direction = two-tailed **then**
 - 11: Mark points where $|T_{obs}(t)| > T_{crit}$ as significant
 - 12: **else if** test direction = one-tailed **then**
 - 13: Mark points where $T_{perm}(t) > T_{crit}$ (or $T_{perm}(t) < -T_{crit}$) as significant (choose direction according to hypothesis)
 - 14: Compute the sum of t-values within each cluster C_{obs} to obtain cluster-level statistics
 - 15: **for** each permutation $p = 1$ to N_{perm} **do**
 - 16: Shuffle condition labels between S_1 and S_2 to create permuted data
 - 17: Compute permuted t-statistics $T_{perm}^p(t)$ for each time point
 - 18: **if** test direction = two-tailed **then**
 - 19: Mark points where $|T_{perm}^p(t)| > T_{crit}$ as significant
 - 20: **else if** test direction = one-tailed **then**
 - 21: Mark points where $T_{perm}^p(t) > T_{crit}$ (or $T_{perm}^p(t) < -T_{crit}$) as significant (choose direction according to hypothesis)
 - 22: Compute the sum of t-values within each permuted cluster $C_{permuted}$
 - 23: **for** each unpermuted cluster C_{obs} **do**
 - 24: Let $N_{cluster}$ be the number of clusters $C_{permuted}$ generated by permutation that have a greater statistics (i.e. sum of t-values) than the cluster C_{obs} we detected in original data
 - 25: **if** $\frac{N_{cluster}}{N_{perm}} < 0.05$ **then**
 - 26: Consider the initial cluster C_{obs} as significant
-

Explanation :

1. Detection of clusters in original data before correction (lines 3-20). Significant time points in the unpermuted data are identified by calculating t-statistics at each time point using MATLAB's *ttest.m* or *ttest2.m* functions. Degrees of freedom (df) are calculated based on the type of comparison:

- For paired comparisons (i.e. condition 1 vs. condition 2), df equals the number of data in the group (e.g., younger or older) in which two conditions are being compared, minus 1.
- For unpaired comparisons (younger vs. older), df equals the number of data available in the younger group for one condition, plus the number of data available for the same condition in the older group, minus 2.

The critical t-value T_{crit} is computed using Student's t inverse cumulative distribution function (MATLAB's *tinvt.m* function) with a significance threshold α of 0.025.

- If our alternative hypothesis H_1 is not oriented (e.g. correctly categorized $\neq$ wrongly categorized), then we mark as significant time points in which $|T_{obs}(t)| > T_{crit}$ (two-tailed t-test).
- If our alternative hypothesis H_1 is oriented however (e.g. recognized $>$ forgotten, incongruent $>$ congruent, etc.), then time points in which $T_{obs}(t) > \pm T_{crit}$ are marked significant, the sign $+$ or $-$ being chosen according to test direction (one-tailed t-test).

All contiguous time points that emerge as significant are grouped into clusters. No minimum duration is imposed.

2. Computation of cluster mass (i.e. the sum of t-values) (line 21). For each cluster identified, the sum of the t-values (i.e., the cluster mass) is calculated. This value represents the average t-value multiplied by the cluster duration. These calculations are repeated on permuted data to allow comparisons between observed and permuted cluster masses.

3. Permutation testing (lines 23-32). We then perform N_{perm} permutations.

- For an unpaired comparison, TEPRs from M participants are combined into a matrix of size M -by- T (where T is the number of time points). For example, assume we have TEPRs from two groups: M_{young} participants in the younger group and M_{older} participants in the

older group, such that $M_{young} + M_{older} = M$. The indices of this combined matrix are randomly shuffled using MATLAB's *randperm.m* function, before being split back into two sub-matrices to simulate independent randomization of group labels.

- For a paired comparison, TEPRs from M participants across two conditions are organized into M -by- T matrices, one for each condition (e.g. if there are two conditions, say recognized and forgotten, then there are two M -by- T matrices). A logical array is created to determine which participants' data should be swapped between the two matrices using MATLAB's *rand.m* function by randomly labeling participants as *true* or *false*. For participants labeled as *true*, their data is swapped between the two matrices, simulating dependent randomization while preserving paired relationships across conditions. Alternatively, one matrix can be formed by subtracting the two conditions, and randomization is performed by multiplying certain rows by -1 (it's mathematically similar).

For each permutation, clusters are identified, and the cluster masses are computed in the same way as for the observed data.

4. Assessing the significance of observed clusters (lines 33-36). For each cluster identified in the original unpermuted data, the cluster mass is compared against the mass of permuted clusters. The number of permuted clusters $N_{cluster}$ with a higher mass than observed clusters in the unpermuted data is calculated. If there are very few of them, or in practice, if $\frac{N_{cluster}}{N_{perm}} < 0.05$, then the cluster we found in the unpermuted is considered statistically significant.

Threshold-free cluster enhancement (TFCE)

TFCE is an alternative to cluster-based permutation testing that avoids the need to choose an arbitrary cluster-forming threshold. Instead TFCE enhances test statistics values according to the extent (duration, for 1D signals) and height (magnitude) of contiguous clustered time points and associated test statistics, producing a TFCE score for every time point. Significance is assessed by permutation testing on the TFCE scores. The following pseudo-code ([Algorithm 3](#)) and explanations adapt TFCE to 1D task-evoked pupillary responses, for paired and unpaired comparisons and for one- or two-tailed hypotheses.

Algorithm 3 TFCE for 1D pupil signal

- 1: **Input:** Pupil signal $S_1(t)$ for condition 1, $S_2(t)$ for condition 2, number of permutations $N_{perm} = 5\,000$, significance level $\alpha = 0.05$, TFCE parameters E and H (for this study: $E = 1$, $H = 2$), integration step Δh (small, e.g. 0.01 of t-range), test direction (one- or two-tailed), data dependence (paired or unpaired)
 - 2: **Output:** Significant time points after permutation testing on TFCE scores
 - 3: Compute observed pointwise test-statistic time-course $T_{obs}(t)$ between S_1 and S_2
 - 4: If test direction = two-tailed, set $A_{obs}(t) = |T_{obs}(t)|$; else (one-tailed), set $A_{obs}(t) = T_{obs}(t)$ (or $-T_{obs}(t)$ if testing negative direction)
 - 5: Compute TFCE score map $TFCE_{obs}(t)$ by applying TFCE transform to $A_{obs}(t)$:
 - 6: For heights h from 0 to $\max(A_{obs})$ with step Δh :
 - 7: Find contiguous intervals (clusters) where $A_{obs}(t) > h$
 - 8: For each time point t inside such intervals, add $\text{extent}(h, t)^E \cdot h^H \cdot \Delta h$ to $TFCE_{obs}(t)$
 - 9: End heights loop
 - 10: Initialize null distribution array M of length N_{perm} to store maximum TFCE values per permutation
 - 11: **for** each permutation $p = 1$ to N_{perm} **do**
 - 12: Create permuted data according to data dependence:
 - 13: **if** unpaired **then**
 - 14: Pool subject TEPRs across groups, shuffle labels, split back into two groups
 - 15: **else if** paired **then**
 - 16: For each subject, randomly swap the two conditions with probability 0.5
 - 17: Compute permuted t-map $T_{perm}^p(t)$ between permuted groups/conditions
 - 18: If test direction = two-tailed, set $A_{perm}^p(t) = |T_{perm}^p(t)|$; else set $A_{perm}^p(t) = \pm T_{perm}^p(t)$
 - 19: Compute $TFCE_{perm}^p(t)$ using the same TFCE parameters and Δh as for observed data (apply TFCE transform to $A_{perm}^p(t)$)
 - 20: Set $M[p] = \max(TFCE_{perm}^p(t))$
 - 21: For each time point t , compute empirical p-value:
 - 22: $p_{value}(t) = \frac{\#\{M[p] \geq TFCE_{obs}(t)\}}{N_{perm}}$
 - 23: Declare time points with $p(t) < \alpha$ significant
-

Explanation :

1. Computation of the test statistic in the original (unpermuted) data (lines 3–4). At each time point, the t-statistic was computed between conditions using MATLAB’s *ttest.m* (for paired data) or *ttest2.m* (for unpaired data). The degrees of freedom (*df*) were defined according to the type of comparison:

- For paired comparisons (e.g., condition 1 vs. condition 2), *df* equals the number of observations within the group being compared (e.g., younger or older participants) minus one.
- For unpaired comparisons (e.g., younger vs. older), *df* equals the sum of the number of observations in each group for a given condition minus two.

2. Computation of the TFCE score in the original (unpermuted) data (lines 5–9).

The observed t-statistic time series was transformed into a TFCE score map (i.e. another time series, since it’s one dimensional). For a range of heights h spanning from 0 to the maximum observed statistic (in steps of Δh), contiguous clusters of time points exceeding h were identified. Each time point t within such clusters was incremented by $\text{extent}(h, t)^E \cdot h^H \cdot \Delta h$, where *extent* refers to the cluster’s temporal duration, and E and H are the TFCE parameters controlling the weighting of cluster extent and height. This integration over h produces a TFCE-enhanced value for each time point, reflecting both the magnitude and the temporal support of the statistical effect.

3. Computation of the TFCE extreme-value null distribution using permuted data (lines 10–20). Permutation testing was used to construct the null distribution of TFCE values. For each permutation, the data were shuffled according to the experimental design:

- For unpaired comparisons, pupil responses were pooled across groups, labels were randomly reassigned, and the data were split back into two pseudo-groups.
- For paired comparisons, the two conditions were randomly swapped within each participant with probability 0.5.

A t-statistic time course was then computed for each permutation, followed by the TFCE transformation using identical parameters as in the original data. The maximum TFCE value across

all time points was stored to form the null distribution of extreme TFCE values.

4. Computation of corrected p-values for each time point (lines 21–23). At each time point, an empirical p-value was derived by comparing the observed TFCE score to the null distribution of maximal TFCE values. Specifically, the p-value corresponds to the proportion of permutations yielding a maximum TFCE score greater than or equal to the observed value. Time points with $p < \alpha$ (here, 0.05) were considered statistically significant, thus controlling the family-wise error rate across time.